

Programming Paradigm :-

- { ① Procedural Programming → C | C++ | Swift | Rust ...
- { ② Object Oriented Programming → C++ | Java | Python |
JavaScript | C#
- ③ Functional Programming → Haskell, Scala, Lisp ...
- ④ Reactive :-

Procedural Programming :-

Procedure :- Set of instructions .



:- Function .

old age name of function / method .

We organize our code into a bunch of
features .

Each procedure may call other procedures
internally .

The execution of a program starts from a specific procedure (called MAIN).

```
a() {  
  | c()  
  }  
b() {  
  | b()  
  }  
c() {  
  | d()  
  }  
main() {  
  | a()  
  }
```

Problems with Procedural Programming :-

Umang is reading chats

Umang is writing notes

Umang is on camera

Students are listening

Scaler is recording the session.

Pattern

Someone is

doing something

Subject + Verb



Entities perform actions

parameters / arguments

```
printStudent (String name, int age, int batch-id) {  
    S.out ( name )  
    S.out (age)  
    S.out (batch-id)  
}
```

Struct / Structure

```
Struct Student {  
    String name,  
    int age,  
    int batch-id  
}
```

Diff b/w struct & class

① Struct can't have method

② All the variables in a struct are visible to anyone else

(everything about an entity is public)

Something / Student
 printStudent (Student s1) {
 | print (s1.name)
 | print (s1.age)
 | print (s1.batch id)
 | }

In Real life,

Someone is doing something

Entities perform action .

In Procedural,

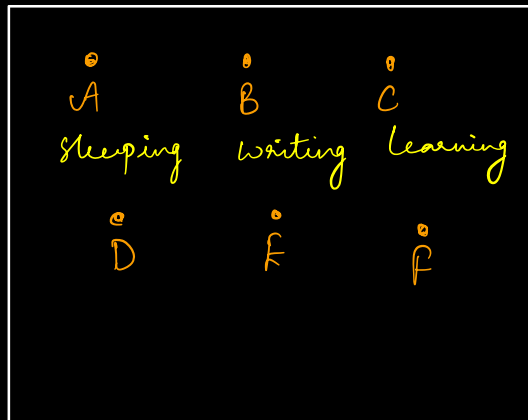
Something is happening on someone

Action is performed on the entity

printStudent (Student) v/s Student.print()

Procedural
World

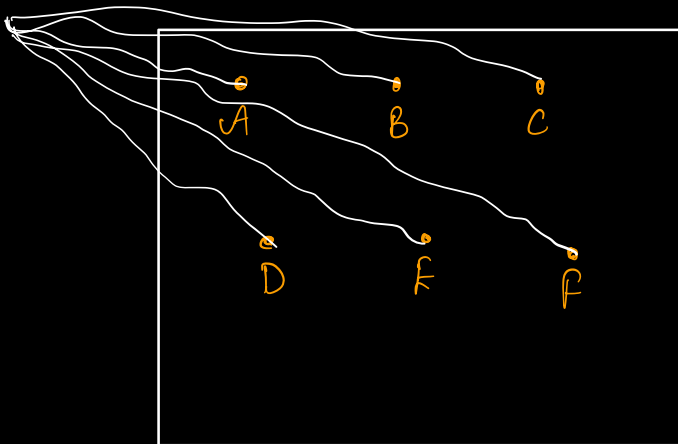
More of a
Real life
Scenario



Controller



Puppet System



Procedural World :-

- 1) Any procedure can play around in any way with attributes of any entity.
- 2) Entities have no free will.

Cons of Procedural Programming:-

- 1) Difficult to make sense of a complex system
- 2) Understanding is difficult-
- 3) Diff to debug
- 4) Spaghetti code:
↳ messy

Object Oriented Programming (OOP) :-

* Entities are core in OOP

* Entities have attributes as well as perform some behaviour.

1 Principle of OOP

3 Pillars of OOP.

Principle :- foundation / concept

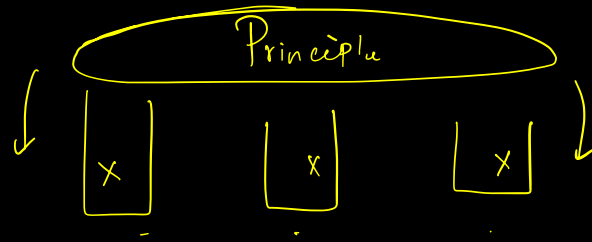
Pillars :- Support

Hold the things together

I will be a good person - My principle.

{ I respect everyone
I will be truthful
I will do hardwork

- My pillars.



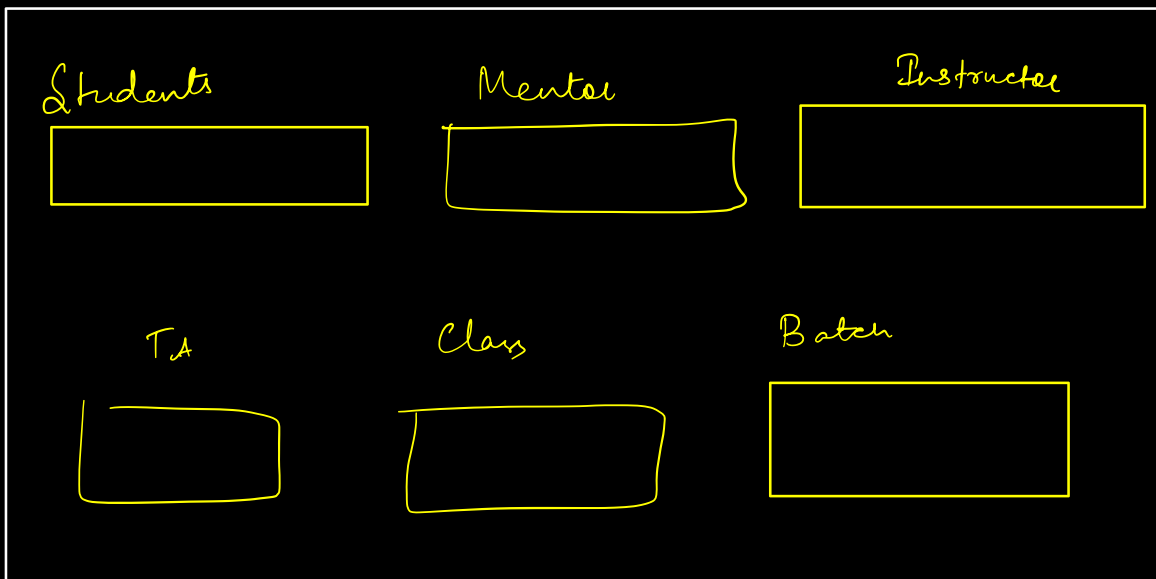
- OOP
- 1 Principle of OOP:- Abstraction ✓
 - 3 Pillars of OOP:- Encapsulation, ✓
Inheritance,
Polymorphism.

Abstraction :- concept of making something abstract.

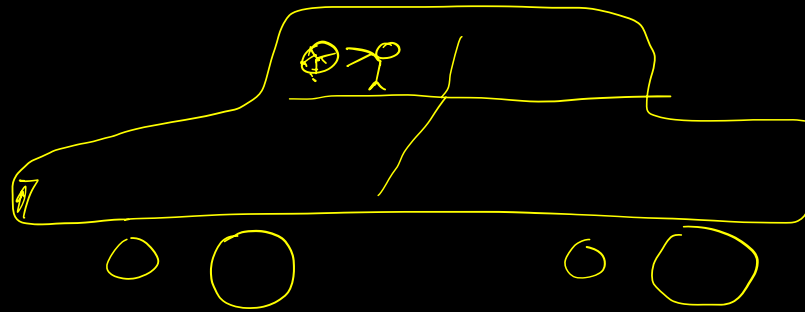
↳ the quality of dealing with ideas.

↳ Represent in terms of ideas / concept.

Idea /
Concept /
Entity } Anything that has attributes (info)
& associated behaviour.



Others don't need to know the details of the idea.



① Representing a complex system in terms of ideas



② Others don't need to know the details of implementation of the idea.

Encapsulation :-

Capsule



Held the medicine together

Protects the medicine from outside environment-

Encapsulation in OOPS:-

→ class

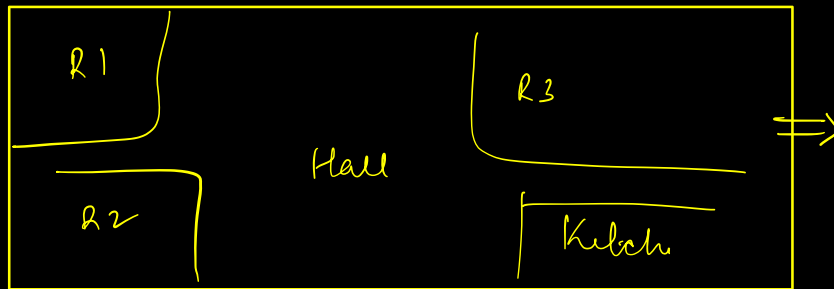
1) Store attributes & behaviour together of an idea.

2) Protect the attributes and behaviour

from illegitimate access from outside.

↳ Next class.

class :- Blueprint of an idea.



└→ Just an idea

class represents the structure of the idea.

- 1) Class takes no space in memory
- 2) Multiple instances of the same class can be created
- 3) Not a Real entity

Object :- Real instance of a class

occupy memory.

each object is completely separate from others.

36089



765

768 —

769 —